

C# Programming Course Syllabus

Archer Infotech, Kothrud Pune | Classroom & live-online batches | Updated September 2026

Course Overview

This Core C# Programming course is designed to build a strong foundation in C# programming, object-oriented programming, .NET fundamentals, exception handling, collections, generics, file handling, LINQ, delegates, events, asynchronous programming, memory management, testing, and practical application development.

The course begins with programming fundamentals and C# syntax, then progresses through classes, objects, inheritance, polymorphism, interfaces, properties, generics, collections, LINQ, delegates, events, file operations, multithreading, async/await, debugging, unit testing, and project development.

The syllabus is suitable for beginners, students, engineering graduates, working professionals, and developers who want to build a strong C# foundation before moving to ASP.NET Core, Web API, Entity Framework Core, Blazor, Azure, desktop development, game development with Unity, or enterprise .NET development.

Course Objectives

By the end of this course, learners will be able to:

- Understand the fundamentals of C# and the .NET platform.
- Write, compile, execute, and debug C# programs.
- Work with variables, data types, operators, conditions, loops, arrays, and strings.
- Apply object-oriented programming principles effectively.
- Design classes, interfaces, namespaces, and reusable components.
- Use properties, indexers, delegates, events, and lambda expressions.
- Handle exceptions and build robust applications.
- Use collections and generics.
- Query data using LINQ.
- Work with files, streams, and serialization.
- Understand asynchronous programming with `async` and `await`.
- Understand basic memory management and garbage collection.
- Write unit tests for C# applications.
- Build console-based mini projects and capstone projects.
- Prepare for C# programming interviews and coding assessments.

Module 1: Introduction to C# and .NET

Topics

- Introduction to Computer Programming
- What is C#?
- History and Evolution of C#
- Features of C#
- Applications of C#
- C# vs C
- C# vs C++
- C# vs Java
- Introduction to .NET
- .NET Framework vs .NET Core vs Modern .NET
- Common Language Runtime
- Common Type System
- Common Language Specification
- Intermediate Language

- Assemblies
- Managed Code
- Unmanaged Code
- Just-In-Time Compilation
- Garbage Collection Overview
- .NET SDK
- .NET Runtime
- C# Compiler
- Introduction to Current .NET Ecosystem

Module 2: Setting Up the C# Development Environment

Topics

- Installing the .NET SDK
- Checking the .NET Version
- dotnet CLI
- Visual Studio
- Visual Studio Code
- JetBrains Rider Overview
- Creating a Console Application
- Project Structure
- .csproj File
- Program.cs
- Build Process
- dotnet new
- dotnet build
- dotnet run
- dotnet clean
- Debug and Release Builds
- NuGet Package Manager Overview
- IDE Debugging Basics

Practical Exercises

- Create Hello World Application
- Run Application Using CLI
- Run Application Using IDE
- Build Debug and Release Versions

Module 3: Structure of a C# Program

Topics

- C# Source Files
- Namespaces
- Classes
- Main() Method
- Top-Level Statements
- Statements
- Code Blocks
- Comments
- Single-Line Comments
- Multi-Line Comments
- XML Documentation Comments
- Keywords

- Identifiers
- Naming Conventions
- Semicolons
- Case Sensitivity
- using Directives
- Coding Style and Formatting

Module 4: Variables, Constants, and Data Types

Topics

- Variable Declaration
- Variable Initialization
- Constants
- const
- readonly Overview
- Value Types
- Reference Types
- Built-In Numeric Types
- byte
- sbyte
- short
- ushort
- int
- uint
- long
- ulong
- float
- double
- decimal
- Character Type
- char
- Boolean Type
- bool
- String Type
- Object Type
- dynamic
- var
- Nullable Value Types
- Nullable Reference Types Overview
- Default Values
- Type Inference
- Type Conversion
- Implicit Conversion
- Explicit Conversion
- Casting
- Convert Class
- Parsing
- TryParse
- Overflow and Underflow
- checked and unchecked

Practical Exercises

- Display Different Data Types

- Temperature Conversion
- Simple Interest Calculator
- Type Conversion Programs
- Parse User Input Safely

Module 5: Operators and Expressions

Topics

- Arithmetic Operators
- Unary Operators
- Relational Operators
- Logical Operators
- Assignment Operators
- Compound Assignment Operators
- Increment and Decrement
- Bitwise Operators
- Shift Operators
- Null-Coalescing Operator ??
- Null-Coalescing Assignment ??=
- Null-Conditional Operators
- Conditional / Ternary Operator
- is Operator
- as Operator
- typeof
- sizeof Overview
- Operator Precedence
- Operator Associativity
- Expressions
- Short-Circuit Evaluation

Practical Exercises

- Even or Odd Number
- Largest of Two Numbers
- Leap Year
- Bitwise Operations
- Null-Coalescing Examples

Module 6: Input and Output

Topics

- Console Applications
- Console.Write()
- Console.WriteLine()
- Console.ReadLine()
- Reading Numeric Values
- Parsing User Input
- TryParse
- Formatted Output
- Composite Formatting
- String Interpolation
- Format Specifiers
- Console Colors Overview
- Input Validation

Practical Exercises

- Student Details Input
- Arithmetic Calculator
- Formatted Invoice
- Mark Sheet
- Validated Numeric Input

Module 7: Decision-Making Statements**Topics**

- if
- if-else
- Nested if
- else-if
- switch
- Switch Statements
- Switch Expressions
- Pattern Matching in Switch
- case
- default
- Conditional Operator
- Best Practices for Decision Logic

Practical Exercises

- Grade Calculator
- Positive, Negative, or Zero
- Menu-Driven Calculator
- Day-of-Week Application
- Electricity Bill Calculator

Module 8: Looping Statements**Topics**

- while
- do-while
- for
- foreach
- Nested Loops
- Infinite Loops
- break
- continue
- Loop Design Best Practices

Practical Exercises

- Multiplication Table
- Sum of Natural Numbers
- Factorial
- Fibonacci Series
- Reverse a Number
- Palindrome Number
- Armstrong Number
- Prime Number
- Number Patterns

- Star Patterns

Module 9: Methods

Topics

- Method Declaration
- Method Definition
- Method Invocation
- Parameters
- Arguments
- Return Types
- void Methods
- Method Overloading
- Optional Parameters
- Named Arguments
- params
- Value Parameters
- Reference Parameters
- ref
- out
- in
- Passing Objects to Methods
- Static Methods
- Instance Methods
- Recursive Methods
- Local Functions
- Expression-Bodied Methods

Practical Exercises

- Function-Based Calculator
- Factorial Using Recursion
- Fibonacci Using Recursion
- GCD and LCM
- Swap Using ref
- Return Multiple Values Using out

Module 10: Arrays

Topics

- Introduction to Arrays
- One-Dimensional Arrays
- Array Declaration
- Array Initialization
- Array Traversal
- Array Length
- Passing Arrays to Methods
- Returning Arrays from Methods
- Multidimensional Arrays
- Rectangular Arrays
- Jagged Arrays
- Array Class
- Sorting Arrays
- Searching Arrays

- Copying Arrays
- Array Utility Methods

Practical Exercises

- Sum and Average
- Maximum and Minimum
- Reverse an Array
- Linear Search
- Binary Search
- Bubble Sort
- Matrix Addition
- Matrix Multiplication
- Matrix Transpose

Module 11: Strings**Topics**

- string Type
- String Immutability
- String Literals
- Verbatim Strings
- Raw String Literals Overview
- String Interpolation
- Comparing Strings
- Important String Methods
- Length
- Substring
- Contains
- IndexOf
- LastIndexOf
- StartsWith
- EndsWith
- Replace
- Trim
- Split
- Join
- ToUpper
- ToLower
- StringBuilder
- String vs StringBuilder
- String Formatting
- String Comparison Options

Practical Exercises

- Reverse a String
- Check Palindrome
- Count Vowels and Consonants
- Character Frequency
- Word Count
- Remove Spaces
- Compare Strings

Module 12: Object-Oriented Programming Fundamentals

Topics

- Procedural Programming vs Object-Oriented Programming
- OOP Principles
- Class
- Object
- Encapsulation
- Abstraction
- Inheritance
- Polymorphism
- Class Definition
- Creating Objects
- Fields
- Methods
- Object State and Behavior
- Access Modifiers
- public
- private
- protected
- internal
- protected internal
- private protected
- this Keyword
- Object References
- Static Members

Practical Exercises

- Student Class
- Employee Class
- Bank Account Class
- Product Class

Module 13: Constructors and Object Initialization**Topics**

- Constructors
- Default Constructor
- Parameterized Constructor
- Constructor Overloading
- Constructor Chaining
- this()
- Static Constructor
- Private Constructor
- Object Initializers
- Collection Initializers
- Constructor Execution Order
- Destructor / Finalizer Overview

Practical Exercises

- Student Constructor Overloading
- Employee Object Initialization
- Constructor Chaining Example

Module 14: Properties and Encapsulation

Topics

- Fields vs Properties
- Property Syntax
- Get Accessor
- Set Accessor
- Auto-Implemented Properties
- Read-Only Properties
- Write-Only Patterns Overview
- Init-Only Properties
- Required Members Overview
- Validation in Properties
- Expression-Bodied Properties
- Encapsulation
- Data Hiding
- Immutable Objects

Practical Exercises

- Bank Account with Properties
- Employee Salary Validation
- Immutable Product Model

Module 15: Indexers**Topics**

- Introduction to Indexers
- Indexer Syntax
- Single-Parameter Indexer
- Multi-Parameter Indexer
- Indexers with Collections
- Indexer vs Property
- Practical Use Cases

Practical Exercises

- Custom Student Collection
- Indexed Inventory Class

Module 16: Inheritance**Topics**

- Introduction to Inheritance
- Base Class
- Derived Class
- base Keyword
- Single Inheritance
- Multilevel Inheritance
- Hierarchical Inheritance
- Constructor Execution in Inheritance
- Protected Members
- Method Hiding
- new Keyword
- Virtual Members
- IS-A Relationship
- HAS-A Relationship

- Composition vs Inheritance

Practical Exercises

- Person and Student
- Employee Hierarchy
- Vehicle Hierarchy
- Shape Hierarchy

Module 17: Polymorphism

Topics

- Compile-Time Polymorphism
- Method Overloading
- Operator Overloading
- Runtime Polymorphism
- Method Overriding
- virtual
- override
- abstract
- Dynamic Binding
- Upcasting
- Downcasting
- Pattern Matching
- Base References
- Polymorphic Collections
- Sealed Overrides

Practical Exercises

- Shape Area Calculation
- Employee Salary Processing
- Payment Processing Hierarchy

Module 18: Abstract Classes

Topics

- Abstraction
- Abstract Classes
- Abstract Methods
- Concrete Methods
- Abstract Properties
- Abstract Classes and Constructors
- Abstract vs Concrete Classes
- When to Use Abstract Classes

Practical Exercises

- Abstract Shape Class
- Abstract Employee Class
- Abstract Vehicle Class

Module 19: Interfaces

Topics

- Interface Basics
- Declaring Interfaces

- Implementing Interfaces
- Multiple Interface Implementation
- Interface Inheritance
- Interface Members
- Default Interface Members Overview
- Explicit Interface Implementation
- Interface vs Abstract Class
- Loose Coupling
- Dependency Inversion Basics
- Programming to Interfaces

Practical Exercises

- Payment Interface
- Printable Interface
- Notification Service Interface
- Multiple Interface Implementation

Module 20: Static, Const, Readonly, and Sealed

Topics

- Static Fields
- Static Methods
- Static Classes
- Static Constructors
- `const`
- `readonly`
- `static readonly`
- `sealed` Classes
- Sealed Methods
- When to Use Each Keyword

Module 21: Structures, Records, and Enumerations

Topics

Structures

- `struct`
- Value Type Behavior
- Struct Constructors
- Struct Properties
- Struct Methods
- Struct vs Class

Records

- Records Overview
- Record Classes
- Record Structs
- Value-Based Equality
- Positional Records
- Immutability Patterns
- `with` Expressions

Enumerations

- `enum`

- Enum Members
- Enum Underlying Types
- Converting Enum Values
- Flags Enum Overview

Practical Exercises

- Coordinate Structure
- Employee Record
- Order Status Enum
- Permission Flags

Module 22: Nullable Types and Null Safety

Topics

- Nullable Value Types
- `Nullable<T>`
- `?` Syntax
- Nullable Reference Types
- Null Forgiving Operator `!`
- Null Conditional Operator `?.`
- Null-Coalescing Operator `??`
- Null-Coalescing Assignment `??=`
- Null Checking Patterns
- Safe Null Handling

Module 23: Exception Handling

Topics

- Error vs Exception
- Exception Hierarchy
- `try`
- `catch`
- Multiple Catch Blocks
- `finally`
- `throw`
- Rethrowing Exceptions
- Custom Exceptions
- Inner Exceptions
- Exception Filters
- Best Practices
- Avoiding Empty Catch Blocks
- Resource Cleanup

Practical Exercises

- Divide-by-Zero Handling
- Invalid Age Exception
- Invalid Balance Exception
- Custom Business Exception
- File Exception Handling

Module 24: Collections Overview

Topics

- Arrays vs Collections
- Generic vs Non-Generic Collections
- Collection Interfaces
- IEnumerable<T>
- ICollection<T>
- IList<T>
- IDictionary<TKey, TValue>
- Iteration
- foreach
- Choosing the Right Collection

Module 25: Generic Collections

Topics

- List<T>
- Dictionary<TKey, TValue>
- HashSet<T>
- SortedSet<T>
- Queue<T>
- Stack<T>
- LinkedList<T>
- SortedDictionary<TKey, TValue>
- SortedList<TKey, TValue>
- Adding Elements
- Removing Elements
- Searching
- Sorting
- Iterating
- Performance Considerations

Practical Exercises

- Student List
- Word Frequency Counter
- Unique Values Using HashSet
- Task Queue
- Undo Stack

Module 26: Generics

Topics

- Introduction to Generics
- Generic Classes
- Generic Methods
- Generic Interfaces
- Type Parameters
- Multiple Type Parameters
- Generic Constraints
- where Clause
- Class Constraints
- Interface Constraints
- Constructor Constraint
- Value-Type Constraint
- Reference-Type Constraint

- Benefits of Generics

Practical Exercises

- Generic Box
- Generic Pair
- Generic Repository
- Generic Maximum Method

Module 27: Delegates

Topics

- Introduction to Delegates
- Delegate Declaration
- Delegate Instantiation
- Invoking Delegates
- Single-Cast Delegates
- Multicast Delegates
- Built-In Delegates
- Action
- Func
- Predicate
- Delegates as Parameters
- Callback Functions
- Method Groups
- Delegates vs Interfaces

Practical Exercises

- Calculator Using Delegates
- Callback Example
- Multicast Notification

Module 28: Lambda Expressions

Topics

- Lambda Syntax
- Expression Lambdas
- Statement Lambdas
- Parameters
- Return Values
- Lambda Variable Capture
- Lambdas with Delegates
- Lambdas with Collections
- Lambdas with LINQ
- Closures Overview

Practical Exercises

- Sort with Lambda
- Filter Data
- Process List Using Lambda

Module 29: Events

Topics

- Introduction to Events

- Publisher-Subscriber Pattern
- event Keyword
- Event Handlers
- Delegates and Events
- Raising Events
- Subscribing to Events
- Unsubscribing
- EventHandler
- Custom EventArgs
- Event Design Best Practices

Practical Exercises

- Temperature Alert System
- Order Status Event
- Notification Publisher-Subscriber

Module 30: LINQ Fundamentals

Topics

- Introduction to LINQ
- LINQ to Objects
- Query Syntax
- Method Syntax
- Where
- Select
- OrderBy
- OrderByDescending
- ThenBy
- GroupBy
- Join
- Distinct
- Take
- Skip
- Any
- All
- Contains
- First
- FirstOrDefault
- Single
- SingleOrDefault
- Count
- Sum
- Average
- Min
- Max
- Aggregate
- Projection
- Filtering
- Grouping
- Joining
- Deferred Execution
- Immediate Execution

Practical Exercises

- Filter Employees by Salary
- Sort Students by Marks
- Group Products by Category
- Join Customer and Order Data
- Calculate Aggregates

Module 31: Advanced LINQ Concepts

Topics

- Anonymous Types
- SelectMany
- Set Operations
- Union
- Intersect
- Except
- Element Operators
- Quantifier Operators
- Conversion Operators
- ToList
- ToArray
- ToDictionary
- Deferred Execution in Depth
- LINQ Performance Basics
- LINQ Best Practices

Module 32: Tuples and Deconstruction

Topics

- Tuple
- Value Tuples
- Named Tuple Elements
- Returning Multiple Values
- Deconstruction
- Discards
- Tuple Comparison
- Use Cases

Practical Exercises

- Return Multiple Calculations
- Swap Values
- Deconstruct Student Data

Module 33: Pattern Matching

Topics

- Type Patterns
- Constant Patterns
- Relational Patterns
- Logical Patterns
- Property Patterns
- Positional Patterns
- List Patterns Overview
- Pattern Matching with `is`

- Pattern Matching with switch
- Switch Expressions

Practical Exercises

- Classify Numeric Values
- Process Different Object Types
- Validate Object Properties

Module 34: File Handling and Streams

Topics

- System.IO
- Files and Directories
- File
- Directory
- FileInfo
- DirectoryInfo
- Paths
- Path
- Streams
- FileStream
- StreamReader
- StreamWriter
- BinaryReader
- BinaryWriter
- Reading Text Files
- Writing Text Files
- Appending Files
- Copying Files
- Moving Files
- Deleting Files
- Creating Directories
- Resource Management with using

Practical Exercises

- Write Student Data to File
- Read Student Data
- Copy a File
- Word and Line Counter
- File-Based Employee Record System

Module 35: Serialization Basics

Topics

- What is Serialization?
- JSON Serialization
- System.Text.Json
- Serializing Objects
- Deserializing Objects
- Serialization Options
- Ignoring Properties
- Custom Property Names
- Working with Collections

- File-Based JSON Persistence
- XML Serialization Overview

Practical Exercises

- Serialize Student Object to JSON
- Deserialize Product List
- Save Application Settings

Module 36: Date and Time

Topics

- DateTime
- DateOnly
- TimeOnly
- TimeSpan
- Current Date and Time
- Parsing
- Formatting
- Date Arithmetic
- Comparing Dates
- UTC vs Local Time
- DateTimeOffset
- Time Zones Overview

Practical Exercises

- Age Calculator
- Date Difference
- Appointment Scheduler
- Format Date and Time

Module 37: Regular Expressions

Topics

- Introduction to Regular Expressions
- Regex
- Character Classes
- Quantifiers
- Groups
- Anchors
- Matching
- Search and Replace
- Splitting Strings
- Input Validation

Practical Exercises

- Email Validation
- Mobile Number Validation
- Password Validation
- Extract Numbers from Text

Module 38: Attributes

Topics

- Introduction to Attributes

- Built-In Attributes
- Obsolete
- Conditional
- Custom Attributes Overview
- Attribute Targets
- Retrieving Attributes
- Use Cases

Module 39: Reflection Basics

Topics

- Introduction to Reflection
- Type
- Inspecting Classes
- Inspecting Properties
- Inspecting Methods
- Inspecting Constructors
- Creating Objects Dynamically Overview
- Reflection Use Cases
- Performance Considerations
- Security Considerations

Module 40: Memory Management and Garbage Collection

Topics

- Managed Memory
- Stack
- Heap
- Value Types
- Reference Types
- Object Allocation
- Garbage Collector
- Generations
- Gen 0
- Gen 1
- Gen 2
- Large Object Heap Overview
- Object Lifetime
- IDisposable
- using
- Finalizers
- Deterministic Cleanup
- Memory Leaks in Managed Applications
- Weak References Overview

Module 41: Multithreading Fundamentals

Topics

- Process vs Thread
- Thread Creation
- Thread Class
- Thread Lifecycle
- Starting Threads

- Sleeping
- Joining
- Background Threads
- Race Conditions
- Shared State
- Thread Safety Basics
- Locking
- lock
- Monitor Overview
- Deadlocks
- Interlocked Operations Overview

Practical Exercises

- Create Multiple Threads
- Thread-Safe Counter
- Shared Resource Demonstration

Module 42: Task Parallel Library

Topics

- Introduction to Task
- Creating Tasks
- Running Tasks
- Waiting for Tasks
- Task Results
- Task<T>
- Task Continuations Overview
- Task.WhenAll
- Task.WhenAny
- Task Parallelism
- Parallel Class Overview
- Cancellation Tokens Overview

Module 43: Asynchronous Programming

Topics

- Synchronous vs Asynchronous Programming
- async
- await
- Task
- Task<T>
- Async Method Design
- Exception Handling in Async Methods
- Async I/O
- Avoiding Blocking Calls
- ConfigureAwait Overview
- Cancellation
- CancellationToken
- Common Async Pitfalls
- Async Best Practices

Practical Exercises

- Simulated Async Download

- Multiple Async Tasks
- Cancel an Async Operation
- Async File Reading

Module 44: Iterators and yield

Topics

- IEnumerable<T>
- IEnumerator<T>
- Iterator Methods
- yield return
- yield break
- Lazy Evaluation
- Custom Iterators
- Iterators and LINQ

Practical Exercises

- Generate Number Sequence
- Filter Data with Iterator
- Custom Collection Iterator

Module 45: Extension Methods

Topics

- Introduction to Extension Methods
- Static Classes
- this Parameter
- Extending Existing Types
- Extension Method Resolution
- Use Cases
- Best Practices

Practical Exercises

- String Extension Methods
- Numeric Utility Extensions
- Collection Extensions

Module 46: Partial Classes and Partial Methods

Topics

- Partial Classes
- Partial Methods
- Splitting Class Definitions
- Generated Code Scenarios
- Practical Use Cases

Module 47: Dependency Management with NuGet

Topics

- Introduction to NuGet
- Package Sources
- Adding Packages
- Removing Packages

- Updating Packages
- Package References
- `dotnet add package`
- Package Versioning
- Package Restore
- Security Considerations

Module 48: Debugging and Error Diagnosis

Topics

- Syntax Errors
- Compilation Errors
- Runtime Errors
- Logical Errors
- Reading Stack Traces
- Visual Studio Debugger
- VS Code Debugging
- Breakpoints
- Conditional Breakpoints
- Step Into
- Step Over
- Step Out
- Watch Windows
- Locals
- Call Stack
- Immediate Window Overview
- Logging Basics
- Debug vs Release Behavior
- Exception Settings

Module 49: Unit Testing Basics

Topics

- Importance of Testing
- Unit Testing Concepts
- Test Cases
- Positive Testing
- Negative Testing
- Boundary Testing
- Regression Testing
- xUnit Overview
- NUnit Overview
- MSTest Overview
- Arrange-Act-Assert Pattern
- Assertions
- Testing Exceptions
- Parameterized Tests
- Testable Code Design
- Mocking Overview
- Test-Driven Development Introduction

Module 50: Project Structure and Solution Management

Topics

- .NET Solutions
- Projects
- `dotnet new sln`
- Adding Projects to Solutions
- Class Libraries
- Project References
- Namespaces
- Layered Folder Structure
- Separation of Concerns
- Reusable Libraries
- Debug vs Release Configuration
- Configuration Files Overview

Module 51: Clean Code and Professional C# Practices**Topics**

- C# Naming Conventions
- PascalCase
- camelCase
- Clean Code Principles
- Small Methods
- Single Responsibility
- Encapsulation
- Immutability
- Null Safety
- Exception Handling Best Practices
- Resource Management
- Async Best Practices
- Choosing Appropriate Collections
- Avoiding Premature Optimization
- Code Reviews
- XML Documentation
- Static Code Analysis Overview
- .NET Analyzers Overview

Module 52: SOLID Principles Introduction**Topics**

- Single Responsibility Principle
- Open/Closed Principle
- Liskov Substitution Principle
- Interface Segregation Principle
- Dependency Inversion Principle
- Coupling and Cohesion
- Composition over Inheritance
- Examples of Poor and Improved Design

Module 53: Basic Design Patterns**Topics**

- Why Design Patterns?
- Singleton Pattern

- Factory Pattern
- Strategy Pattern
- Observer Pattern
- Repository Pattern Overview
- Dependency Injection Concept
- When to Use and Avoid Patterns

Module 54: Data Structures Using C#

Topics

- Arrays
- Lists
- Linked Lists
- Stacks
- Queues
- Hash Tables
- Sets
- Dictionaries
- Trees Overview
- Binary Search Tree Introduction
- Graphs Overview
- Choosing the Right Data Structure

Algorithms

- Linear Search
- Binary Search
- Bubble Sort
- Selection Sort
- Insertion Sort
- Merge Sort Overview
- Quick Sort Overview

Module 55: Problem-Solving and Algorithmic Thinking

Topics

- Understanding Problem Statements
- Identifying Inputs and Outputs
- Constraints
- Pseudocode
- Dry Runs
- Edge Cases
- Time Complexity Introduction
- Space Complexity Introduction
- Big-O Notation Basics
- Recursion
- Searching
- Sorting
- Common Problem-Solving Patterns
- Choosing Appropriate Collections

Practice Categories

- Number Problems
- Pattern Problems

- Array Problems
- String Problems
- Collection Problems
- OOP Problems
- LINQ Problems
- Recursion Problems
- File Handling Problems

Module 56: Git and GitHub for C# Developers

Topics

- Introduction to Version Control
- Installing Git
- Creating a Repository
- `git init`
- `git add`
- `git commit`
- `git status`
- `git log`
- Branching Basics
- Merging
- GitHub Repository Creation
- Push and Pull
- Pull Requests Overview
- README Documentation
- `.gitignore`
- .NET-Specific `.gitignore`
- Building a Programming Portfolio

Module 57: AI-Assisted C# Development

Topics

- Using AI Coding Assistants Responsibly
- Generating Boilerplate Code
- Explaining C# Code
- Understanding Compiler Errors
- Debugging with AI Assistance
- Generating Unit Tests
- Refactoring Suggestions
- LINQ Optimization Suggestions
- Async/Await Review
- Code Review with AI
- Generating Documentation
- Security Risks of Blindly Using Generated Code
- Validating AI-Generated Solutions
- Prompting Techniques for C# Programming

Recommended Practice

Students should learn to:

- Ask AI for explanations instead of only solutions.
- Review generated code manually.
- Run tests and validate edge cases.
- Check async and thread-safety assumptions.

- Verify LINQ queries.
- Review nullability warnings.
- Understand every generated line before submission.

Module 58: Mini Projects

Students should complete multiple mini projects such as:

- Calculator Application
- Student Grade Management System
- Employee Management System
- Library Management System
- Banking Management System
- Inventory Management System
- Contact Management System
- Quiz Application
- Expense Tracker
- Task Management Application
- Hotel Reservation System
- Vehicle Rental System
- File-Based Address Book
- JSON-Based Student Management System
- Employee Payroll System
- Console-Based E-Commerce Cart
- Banking Application with Exception Handling
- LINQ-Based Reporting Application

Module 59: Capstone Project

Learners should build one complete C# application using the concepts covered in the course.

Suggested Capstone Projects

- Library Management System
- Hospital Management System
- Student Information System
- Banking Management System
- Employee Payroll System
- Inventory and Billing System
- Hotel Reservation System
- Vehicle Rental System
- Task Management System
- Console-Based CRM Application

Capstone Requirements

The project should demonstrate:

- Object-oriented design
- Classes and objects
- Encapsulation
- Inheritance where appropriate
- Polymorphism
- Interfaces
- Properties
- Generics
- Collections

- LINQ
- Delegates and events where appropriate
- Exception handling
- File handling or JSON persistence
- Async programming where appropriate
- Input validation
- Modular code organization
- Unit testing basics
- Git-based version control
- README documentation

Module 60: Core C# Interview Preparation

Core C# Interview Topics

- C# and .NET Architecture
- CLR
- CTS and CLS
- Managed vs Unmanaged Code
- Value Types vs Reference Types
- Stack vs Heap
- Boxing and Unboxing
- var vs dynamic vs object
- String Immutability
- String vs StringBuilder
- Classes and Objects
- Constructors
- Properties
- Indexers
- Inheritance
- Polymorphism
- Abstraction
- Interfaces
- Abstract Class vs Interface
- Method Overloading vs Overriding
- virtual, override, and new
- static
- const
- readonly
- Struct vs Class
- Record vs Class
- Nullable Types
- Exception Handling
- Collections
- Generics
- List vs Array
- Dictionary vs Hashtable
- Delegates
- Events
- Lambda Expressions
- LINQ
- Deferred Execution
- IEnumerable<T>
- IQueryable<T> Overview

- File Handling
- Serialization
- Garbage Collection
- IDisposable
- using
- Threads
- Tasks
- Async/Await
- ref, out, and in
- Extension Methods
- Reflection
- SOLID Principles

Interview Activities

- Conceptual Questions
- Output-Based Questions
- Debugging Questions
- Coding Challenges
- Collection-Based Problems
- LINQ Problems
- OOP Design Questions
- Async Programming Questions
- Mock Interviews
- Timed Coding Tests

Recommended Lab Assignments

Learners should complete at least:

- 20 Basic Syntax and Operator Programs
- 20 Conditional and Looping Programs
- 15 Method and Recursion Programs
- 20 Array and String Programs
- 20 OOP Programs
- 10 Inheritance and Polymorphism Programs
- 10 Interface and Abstraction Programs
- 15 Exception Handling Programs
- 20 Collections and Generics Exercises
- 20 LINQ and Lambda Exercises
- 10 File Handling Programs
- 5 Multithreading / Async Exercises
- 5 Unit Testing Exercises
- 5 Mini Projects
- 1 Capstone Project

Suggested Course Duration

- Section — Suggested Duration
- C# and .NET Fundamentals — 12–15 Hours
- Control Flow, Methods, Arrays, and Strings — 18–22 Hours
- Object-Oriented Programming — 22–28 Hours
- Exceptions, Generics, and Collections — 18–22 Hours
- Delegates, Events, Lambdas, and LINQ — 18–22 Hours
- File Handling, Serialization, and Utilities — 10–12 Hours
- Multithreading and Async Programming — 12–15 Hours

- Testing, SOLID, and Code Quality — 10–12 Hours
- Data Structures and Problem Solving — 12–15 Hours
- Projects and Interview Preparation — 20–25 Hours

Recommended Total Duration: approximately **150–185 hours**, depending on practical depth, assignments, and project work.

Prerequisites

No prior programming experience is required.

Basic familiarity with computers, files, folders, and command-line usage is helpful.

Learners who already know C, C++, or Java may progress faster through the introductory programming modules.

Tools and Technologies

- C#
- .NET SDK
- .NET CLI
- Visual Studio
- Visual Studio Code
- JetBrains Rider
- NuGet
- Git
- GitHub
- xUnit / NUnit / MSTest
- System.Text.Json
- Command Line / Terminal

Learning Outcomes

After completing the course, learners should be able to:

- Write structured and object-oriented C# applications.
- Understand .NET execution and CLR fundamentals.
- Apply encapsulation, inheritance, polymorphism, and abstraction.
- Use interfaces and namespaces to design modular applications.
- Work with properties, indexers, delegates, and events.
- Handle errors using exception handling.
- Work effectively with collections and generics.
- Query and transform data using LINQ.
- Perform file operations and JSON serialization.
- Build asynchronous applications using async/await.
- Understand managed memory and garbage collection.
- Debug and test C# applications.
- Use Git and GitHub professionally.
- Build portfolio-ready C# projects.
- Solve Core C# interview and coding problems.
- Progress confidently to enterprise .NET development.

Recommended Progression After Core C#

After completing Core C#, learners can continue with:

- Advanced C#
- ASP.NET Core

- ASP.NET Core MVC
- ASP.NET Core Web API
- Entity Framework Core
- SQL Server
- Dependency Injection in Depth
- Authentication and Authorization
- Microservices with .NET
- Blazor
- SignalR
- Azure for .NET Developers
- Docker and Kubernetes
- Clean Architecture
- Design Patterns
- Unit and Integration Testing
- .NET MAUI
- Unity Game Development
- Data Structures and Algorithms in C#
- System Design for .NET Developers

Enquire or book a free demo session. Archer Infotech, Flat No. 12, Divyadarshan Housing Society, behind Kothrud Bus Stand Road, Kothrud, Pune 411038. Phone +91 9850 678451. Email info@archerinfotech.in. Web archerinfotech.in.